

CSI CYBER

INTRODUÇÃO A MALWARE ANALYSIS



2

LISTA DE FIGURAS

Figura 1 – Site para download do Virtualbox.....	8
Figura 2 - Instalação do Virtualbox.....	9
Figura 3 – Versão do Virtualbox.....	10
Figura 4 – Instalação do Virtualbox modo 2.....	11
Figura 5 – Versão do Virtualbox modo 2.....	11
Figura 6 – Arquivo de instalação do Virtualbox para Windows.....	12
Figura 7 – Baixar imagem ISO do Windows.....	13
Figura 8 – Baixar imagem ISO do Ubuntu.....	13
Figura 9 – Baixar imagem ova do REMnux.....	14
Figura 10 – Configurando nome da VM Ubuntu.....	14
Figura 11 – Configurando quantidade de memória da VM Ubuntu.....	15
Figura 12 – Configurando disco da VM Ubuntu.....	15
Figura 13 – Configurando tipo de disco da VM Ubuntu.....	16
Figura 14 – Configurando a dinâmica do disco da VM Ubuntu.....	16
Figura 15 – Configurando o tamanho e nome do disco da VM Ubuntu.....	17
Figura 16 – Criação da VM Ubuntu concluída.....	17
Figura 17 – Configuração da imagem ISO.....	18
Figura 18 – Configuração da imagem ISO.....	18
Figura 19 – Configuração da imagem ISO.....	19
Figura 20 – Conversão de binário para decimal.....	23
Figura 21 – Conversão de binário para hexadecimal.....	25
Figura 22 – Tabela ASCII convertida em decimal, binário e hexa.....	27
Figura 23 – Verificação do arquivo mydocs.pdf no vírus total.....	33

LISTA DE TABELAS

Tabela 1 – Conversão direta entre as bases apresentadas	22
Tabela 2 – Conversão de decimal para binário	23
Tabela 3 – Tabela auxiliar de conversão binário decimal.....	24
Tabela 4 – Conversão direta entre binário e hexadecimal	24
Tabela 5 – Conversão de decimal para hexadecimal.....	25

EXEMPLO

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Verificação do arquivo de instalação do Virtualbox	9
Comando de prompt 2 – Instalação do Virtualbox.....	10
Comando de prompt 3 – Instalação do Virtualbox modo 2.....	10
Comando de prompt 4 – Utilizando o comando echo e o comando de hd do Linux.....	27
Comando de prompt 5 – Levantando informações sobre um arquivo	28
Comando de prompt 6 – Levantando Informações do arquivo mydocs2.pdf	29
Comando de prompt 7 – Localização do arquivo magic.mgc	30
Comando de prompt 8 – Instalação do hexer	30
Comando de prompt 9 – Magic number de um arquivo .pdf	31
Comando de prompt 10 – Conferindo o arquivo antes da edição	31
Comando de prompt 11 – Abrindo o arquivo mydocs.txt editor de hexadecimal.....	31
Comando de prompt 12 – Edição do arquivo mydocs.pdf com o hexer	31
Comando de prompt 13 – Adição do Magic number ao arquivo mydocs.pdf	31
Comando de prompt 14 – Arquivo de texto mascarado em um pdf	32
Comando de prompt 15 – Assinatura de um arquivo executável	32
Comando de prompt 16 – Assinatura filtrada do arquivo executável	32
Comando de prompt 17 – Inserção do magic number executável	33
Comando de prompt 18 – Verificação do arquivo mydocs.pdf no linux.....	33

SUMÁRIO

1 INTRODUÇÃO A MALWARE ANALYSIS	6
1.1 Introdução	6
2 MÁQUINA VIRTUAL	7
3 VIRTUALBOX.....	8
3.1 Instalação do Virtualbox	8
3.1.1 Linux modo 1.....	8
3.1.2 Linux modo 2.....	10
3.1.3 Windows.....	11
3.2 Criando a VM	12
4 SISTEMAS DE NUMERAÇÃO	19
4.1 Sistema decimal	19
4.2 Sistema Hexadecimal.....	20
4.3 Sistema Binário	21
4.4 Conversão entre bases	21
4.5 Conversão de decimal para binário	22
4.6 Conversão de binário para decimal	23
4.7 Conversão de binário para hexadecimal	24
4.8 Conversão de decimal para hexadecimal.....	25
5 TABELA ASCII	26
5.1 ASCII extendida	28
6 ARQUIVOS	28
CONCLUSÃO.....	34
REFERÊNCIAS.....	35

1 INTRODUÇÃO A MALWARE ANALYSIS

1.1 Introdução

Em um dia típico de sexta-feira você inicia sua rotina diária para finalização do expediente, guarda livros, notebook na mochila, fecha vários e vários terminais abertos durante o dia para manutenção e monitoramento de logs dos servidores, então às 17:45h recebe a primeira ligação:

Usuário Financeiro:

– Aqui é do financeiro e a planilha do pagamento do próximo mês não está abrindo, o que eu faço?

Gerente de Redes:

– Ok, acionarei alguém do suporte para que seja resolvido o problema.

Suporte:

– Chefe, o problema não é apenas com uma planilha, mas todos os arquivos do computador como fotos, doc, pdf, músicas também não abrem, e agora apareceu uma mensagem informando que os dados foram sequestrados e pedem um valor de resgate o que eu faço?

Gerente de Redes:

– Ok, como temos um backup de ontem em nosso servidor, pegue a máquina para ser formatada e amanhã copiaremos o backup para o usuário e assim o problema será resolvido.

Então, às 17h50 recebe outra chamada e outros telefones da gerência de redes começam a tocar, e o que seria um dia típico de sexta com *happy hour* se transforma em um *black day* que não terá hora para acabar.

Cenários como este não são incomuns de ocorrer e, na realidade, a probabilidade de ocorrerem tem crescido exponencialmente conforme as ameaças estão mais modeladas para *bypass* sistemas complexos de proteção. E, são em casos como estes que entendemos a importância das habilidades humanas (*soft skills*), que mesmo as melhores Inteligências Artificiais ainda não foram capazes de alcançar.

Durante esta disciplina você será capacitado a fazer frente às principais ameaças modernas, conhecendo diversas ferramentas, tipos de malwares, tipos de análises, bem como o comportamento de um malware, tanto estática quanto dinamicamente, além de praticar alguns tipos mais comuns de análise de malware.

Ao final deste primeiro capítulo, estaremos com o nosso ambiente de laboratório pronto e poderemos iniciar nossa incrível jornada em Análise de Malwares.

2 MÁQUINA VIRTUAL

Para iniciar nossa disciplina, faz-se necessário um ambiente previamente configurado e pronto para o uso utilizando-se de máquinas virtuais.

Por que usar uma máquina virtual? A utilização de máquina virtual (VM) nos dará um ganho de tempo, pois existe a necessidade de executar os malwares para que possamos analisá-los dinamicamente e estaticamente, porém, não é interessante executarmos em nosso sistema operacional (host) instalado no computador, pois, dependendo do malware, seria necessário uma reinstalação do S.O. e de todas as ferramentas necessárias, e isso demandaria um bom tempo.

Com uso da VM, podemos criar snapshots para que possamos voltar a um estado anterior da VM sem ter que fazer toda reinstalação. Além do ganho de tempo, não nos preocuparemos com nossos arquivos pessoais (fotos, pdf, doc ...), pois estaremos isolados da VM em que analisamos o malware, e nossos arquivos estarão em nosso host.

DICA: É interessante que, quando possível, o host não execute o mesmo S.O da VM que iremos analisar o malware.

Exemplo: Para analisar um malware em uma VM Windows é interessante que estejamos em um host Linux, ou vice-versa.

No entanto, diferentemente de todos os outros sistemas operacionais, essas máquinas virtuais não são máquinas estendidas, com arquivos e outros aspectos interessantes. Em vez disso, elas são cópias exatas do hardware exposto, incluindo modos núcleo/usuário, E/S, interrupções e tudo mais que a máquina tem. Como cada máquina virtual é idêntica ao hardware original, cada uma delas pode executar qualquer sistema operacional capaz de ser executado diretamente sobre o hardware (TANENBAUM; BOS, 2016, p. 48).

3 VIRTUALBOX

O Virtualbox é um poderoso sistema de virtualização x86/AMD64 que é utilizado por empresas e por usuários finais. Com o Virtualbox poderemos criar snapshots das VMs, isolar as VMs em redes (host-only), criar clones (linkados ou completos) etc. O Virtualbox é rico em recursos e faz tudo que outros softwares de virtualização fazem, com apenas um detalhe, é gratuito e licenciado como Software de Código Aberto sob GPL2.

3.1 Instalação do Virtualbox

3.1.1 Linux modo 1

A instalação do Virtualbox no Linux é extremamente simples e rápida podendo ser realizada de duas maneiras: baixando a versão mais atualizada no site do Virtualbox ou instalando diretamente dos repositórios em nossa distribuição. Focarei na distribuição Ubuntu LTS 18.04 (Debian like), pois é a versão que utilizaremos em nossa disciplina.

Vamos começar com a instalação a partir do site do Virtualbox. Devemos baixar a última versão em <https://www.virtualbox.org/wiki/Linux_Downloads>.



Figura 1 – Site para download do Virtualbox
Fonte: Virtualbox (2019)

Após ter feito o download do Virtualbox, utilizaremos a linha de comando para fazer a verificação do arquivo baixado, ou seja, se o arquivo que baixamos é exatamente igual ao arquivo do site do Virtualbox e, também, a sua instalação, como visto na figura “Verificação do arquivo de instalação do Virtualbox”.

COMANDO:

```
cd /home/hugo/Downloads
```

```
ls
```

```
wget https://www.virtualbox.org/download/SHA256SUMS -q
```

```
sha256sum -c SHA256SUMS --ignore-missing
```

Comando de prompt 1 – Verificação do arquivo de instalação do Virtualbox
Fonte: Elaborado pelo autor (2020)

DICA: Importante observar os seguintes detalhes: quando no prompt de comando existir o símbolo “\$” estaremos utilizando o usuário sem privilégios administrativos e quando for o “#” estaremos executando com o usuário root. Apenas o usuário root tem os privilégios necessários para a instalação de pacotes e softwares no Linux.

Depois de ter verificado a integridade do arquivo, partiremos para a instalação propriamente dita. Utilizaremos apenas um comando para a instalação, como na figura “Instalação do Virtualbox”.

```
root@ubuntu:/home/hugo/Downloads# apt install ./virtualbox-6.1.12-139181-Ubuntu-bionic_amd64.deb
Reading package lists... Done
Building dependency tree
Reading state information... Done
Note, selecting 'virtualbox-6.1' instead of './virtualbox-6.1.12-139181-Ubuntu-bionic_amd64.deb'
The following packages were automatically installed and are no longer required:
  dkms efibootmgr libegl1-mesa libfwup1 libgsoap-2.8.60 liblvm9 libvncserver1 linux-headers-5.3.0-61 linux-headers-5.3.0-61-generic
  linux-image-5.3.0-61-generic linux-modules-5.3.0-61-generic linux-modules-extra-5.3.0-61-generic virtualbox-dkms
Use 'apt autoremove' to remove them.
The following additional packages will be installed:
```

Saída omitida

```
Processing triggers for man-db (2.8.3-2ubuntu0.1) ...
Processing triggers for shared-mime-info (1.9-2) ...
Processing triggers for gnome-menus (3.13.3-11ubuntu1.1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for mime-support (3.60ubuntu1) ...
root@ubuntu:/home/hugo/Downloads# exit
```

Figura 2 - Instalação do Virtualbox
Fonte: Elaborado pelo autor (2020)

COMANDOS

```
apt install ./virtualbox-6.1.12-139181-ubuntu-bionic_amd64.deb
```

```
exit
```

Comando de prompt 2 – Instalação do Virtualbox
Fonte: Elaborado pelo autor (2020)

Após finalizar a instalação, vamos verificar se ocorreu tudo conforme esperado, abrindo o Virtualbox e conferindo sua versão clicando em “Ajuda”; e depois, em “Sobre Virtualbox”, como vemos na figura “Versão do Virtualbox”.



Figura 3 – Versão do Virtualbox
Fonte: Elaborado pelo autor (2020)

3.1.2 Linux modo 2

Agora utilizaremos os repositórios do Ubuntu para fazer a instalação do Virtualbox. Aconselho a todos utilizarem este método, pois além de ser mais fácil, o Virtualbox instalado já foi testado pela comunidade Ubuntu.

Execute o comando, como na figura “Instalação do Virtualbox modo 2”, responda sim para continuar com a instalação.

COMANDOS

```
apt install virtualbox
```

```
Y
```

Comando de prompt 3 – Instalação do Virtualbox modo 2

Fonte: Elaborado pelo autor (2020)

```
root@ubuntu:~# apt install virtualbox
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  efibootmgr libcurl4 libegl1-mesa libfwup1 libllvm9 libstdc++-5.3.0-61
  linux-headers-5.3.0-61-generic linux-image-5.3.0-61-generic linux-modules-5.3.0-61-generic
  linux-modules-extra-5.3.0-61-generic
Use 'apt autoremove' to remove them.
The following additional packages will be installed:
  virtualbox-qt
Suggested packages:
  vde2 virtualbox-guest-additions-iso
The following NEW packages will be installed:
  virtualbox virtualbox-qt
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 25.9 MB of archives.
After this operation, 109 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
```

Figura 4 – Instalação do Virtualbox modo 2

Fonte: Elaborado pelo autor (2020)

Assim como no 1º modo de instalação, vamos verificar qual a versão do Virtualbox baixado e instalado do repositório do Ubuntu na figura “Versão do Virtualbox modo 2”.



Figura 5 – Versão do Virtualbox modo 2

Fonte: Elaborado pelo autor (2020)

3.1.3 Windows

Para a instalação do Virtualbox no Windows 10 precisaremos apenas baixar o arquivo de instalação em <<https://www.virtualbox.org/wiki/Downloads>>, clicando em Windows hosts para baixar a última versão do Virtualbox e executá-lo.



Figura 6 – Arquivo de instalação do Virtualbox para Windows
Fonte: Elaborado pelo autor (2020)

3.2 Criando a VM

Para iniciar a análise de um malware no Virtualbox, precisamos de um ambiente seguro e protegido, pois não queremos que nosso sistema ou nossa rede sejam infectados. O seu laboratório dependerá muito dos recursos disponíveis. Por exemplo, iniciar um Windows 10 numa máquina virtual com 1 GB de memória RAM e 1 processador, dependendo da ferramenta utilizada, talvez não tenha o mesmo desempenho utilizando 4GB de RAM e 4 processadores. Em contrapartida, hoje, a maioria dos notebooks ou PCs vem com 8 GB de RAM, então, não é o caso reservar a metade dos seus recursos apenas para uma máquina virtual, pois algumas vezes teremos, virtualizadas, 2 máquinas ao mesmo tempo. A criação das VMs será de acordo com os recursos disponíveis de cada um.

Configurarei apenas como exemplo: primeiramente, criaremos uma VM com Ubuntu 18.04 para que possamos executar alguns comandos deste capítulo de introdução e alguns outros para explicações em linha de comando, quando iniciarmos a análise estática e dinâmica serão necessárias mais 2 VMs, uma com Windows para executar o malware e outra com a distribuição REMnux.

DICA: Não será abordada a instalação dos sistemas operacionais Windows e Linux, apenas como configurar uma VM para a instalação deles.

Antes de continuarmos, vamos baixar as imagens .iso para que possamos fazer a instalação dos sistemas.

Download do Windows em: <<https://www.microsoft.com/pt-br/software-download/windows10ISO>>.

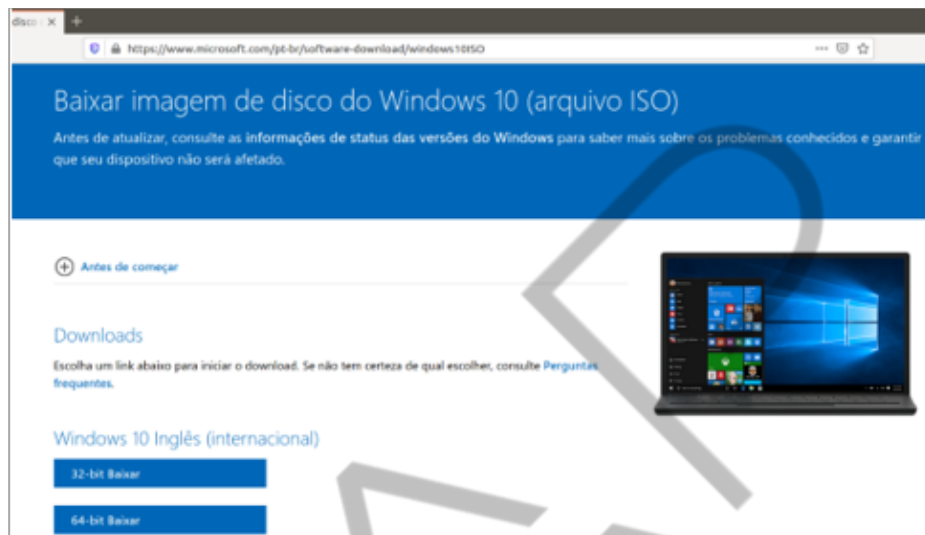


Figura 7 – Baixar imagem ISO do Windows
Fonte: Microsoft (2020)

Download do Ubuntu em: <<https://releases.ubuntu.com/bionic/>>.

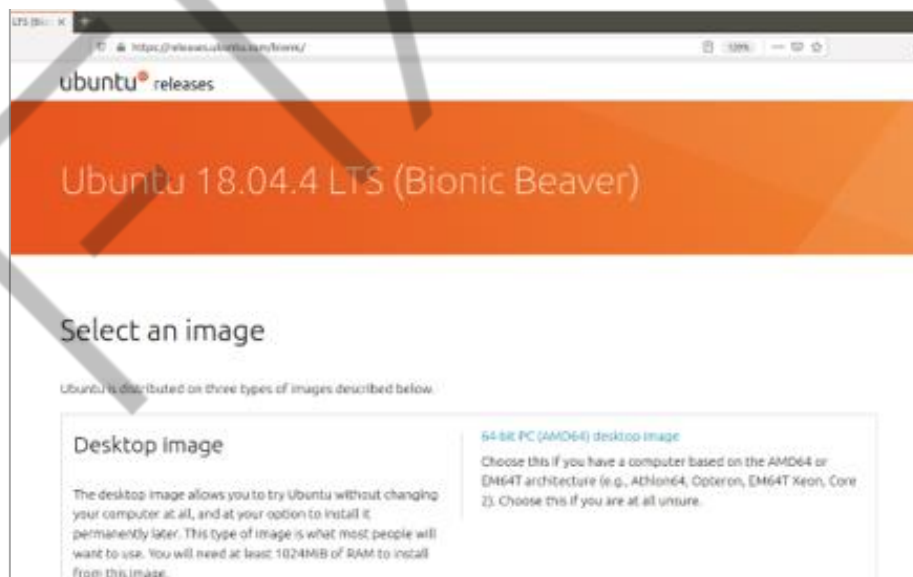


Figura 8 – Baixar imagem ISO do Ubuntu
Fonte: Ubuntu (2020)

Download do REMnux, para Virtualbox, em: <<https://docs.remnux.org/install-distro/get-virtual-appliance>>.

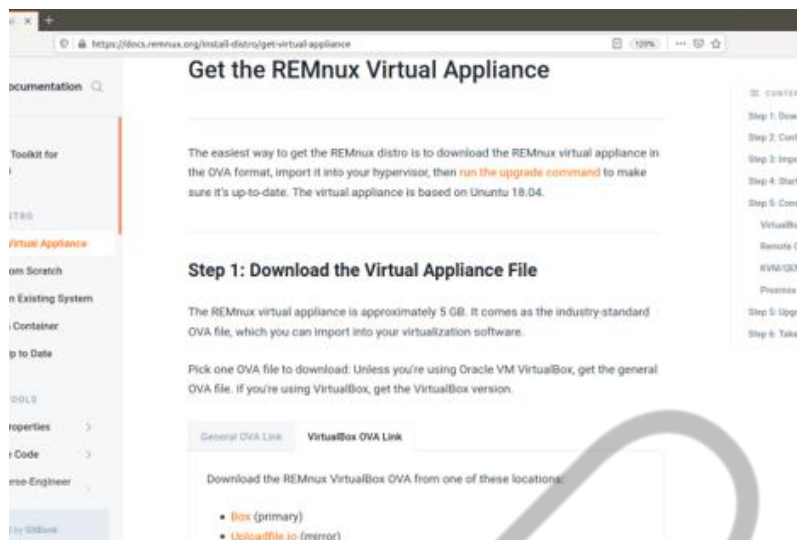


Figura 9 – Baixar imagem ova do REMnux
Fonte: REMnux (2020)

Após concluir nossos downloads, iniciaremos a configuração e instalação de nossas VMs. As configurações das VMs Windows e Ubuntu serão, inicialmente, idênticas, porém a configuração do REMnux será um pouco diferente em relação às outras.

Criação e configuração da VM Ubuntu

Clicando em novo, a janela da figura “Configurando nome da VM Ubuntu” abaixo se abrirá para preenchimento do nome da VM, depois escolha o tipo e a versão do sistema operacional que esteja instalando.



Figura 10 – Configurando nome da VM Ubuntu
Fonte: Elaborado pelo autor (2020)

Após configurado o nome, vamos para a configuração da memória RAM, no meu caso deixarei com o padrão, 1024 MB.



Figura 11 – Configurando quantidade de memória da VM Ubuntu
Fonte: Elaborado pelo autor (2020)

A VM precisará ser armazenada em algum disco, então o Virtualbox criará um arquivo de disco rígido para este armazenamento, como na figura “Configurando disco da VM Ubuntu”.



Figura 12 – Configurando disco da VM Ubuntu
Fonte: Elaborado pelo autor (2020)

Agora escolheremos o tipo de disco rígido que criaremos, podemos deixar como o padrão do Virtualbox.



Figura 13 – Configurando tipo de disco da VM Ubuntu

Fonte: Elaborado pelo autor (2020)

No passo abaixo, é interessante marcar como dinamicamente alocado, pois o tamanho do arquivo de disco aumentará à medida que arquivos forem gravados na VM, caso escolha tamanho fixo, o arquivo de disco terá um tamanho previamente fixado. Ex.: Caso a instalação de sua VM ocupe 10 GB de espaço, mas você optou por um tamanho fixo de arquivo de disco de 32 GB, então este arquivo ocupará 32 GB de espaço em seu HD, porém, caso escolha um dinamicamente alocado, esse arquivo apenas ocupará pouco mais de 10 GB em seu disco rígido.



Figura 14 – Configurando a dinâmica do disco da VM Ubuntu

Fonte: Elaborado pelo autor (2020)

Agora vamos escolher o tamanho e o nome para nosso arquivo de disco.



Figura 15 – Configurando o tamanho e nome do disco da VM Ubuntu
Fonte: Elaborado pelo autor (2020)

Após configurado o disco, terminamos a configuração inicial da VM e temos a imagem “Criação da VM Ubuntu concluída”.

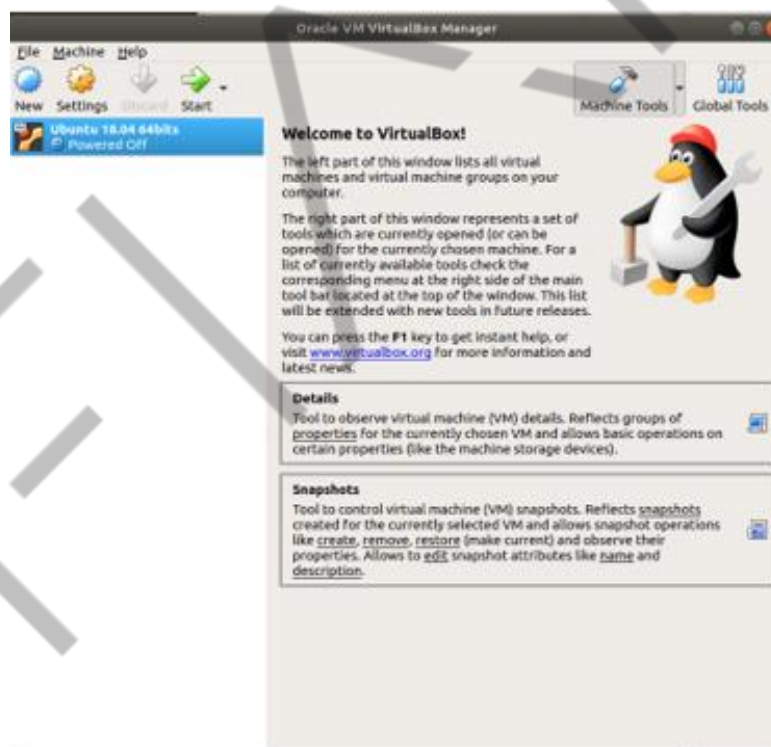


Figura 16 – Criação da VM Ubuntu concluída
Fonte: Elaborado pelo autor (2020)

Agora é necessário que indiquemos qual imagem ISO nossa VM utilizará para iniciar a instalação do sistema operacional, como na figura “Configuração da imagem ISO”, e escolha a imagem ISO baixada anteriormente.

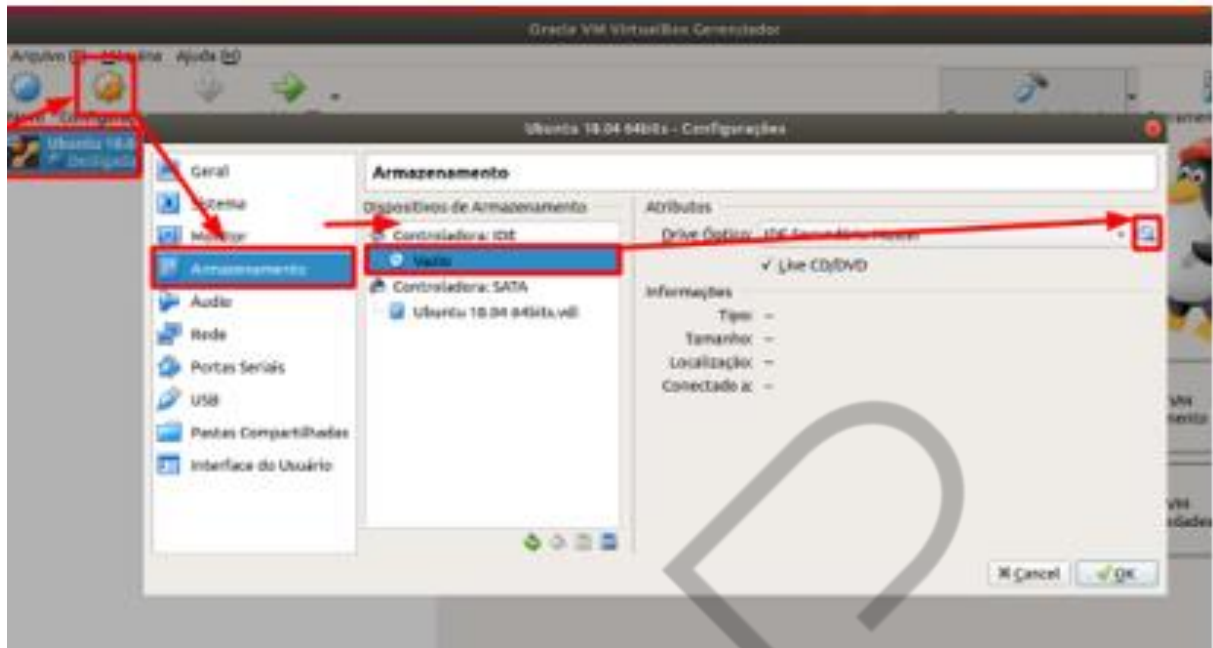


Figura 17 – Configuração da imagem ISO
Fonte: Elaborado pelo autor (2020)

Clique em “Selecionar Arquivo de Disco Óptico Virtual” e escolha a imagem iso do Ubuntu.

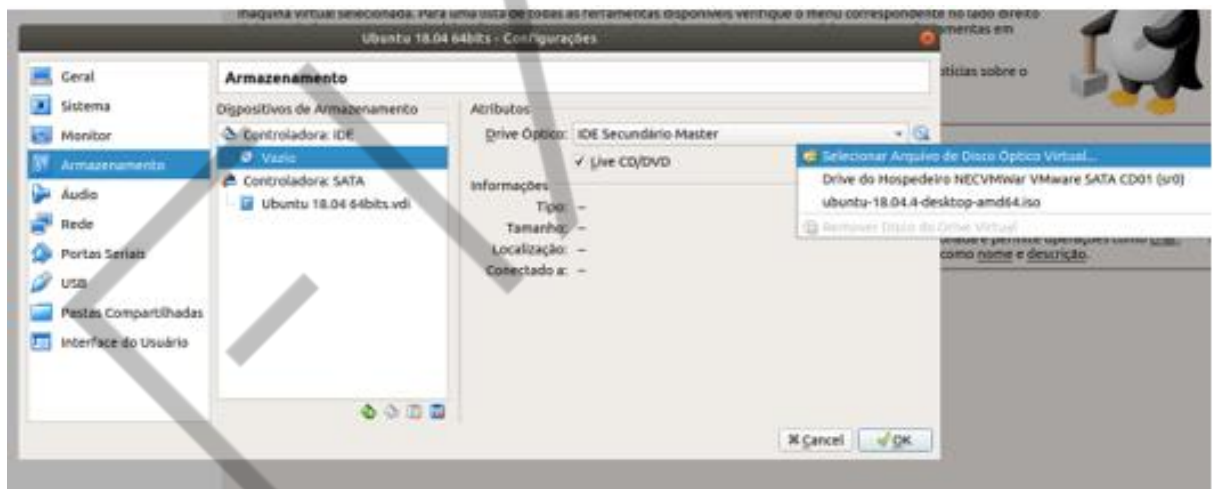


Figura 18 – Configuração da imagem ISO
Fonte: Elaborado pelo autor (2020)

Após configurada a imagem, iniciaremos a VM para a conclusão do processo de instalação do sistema operacional.

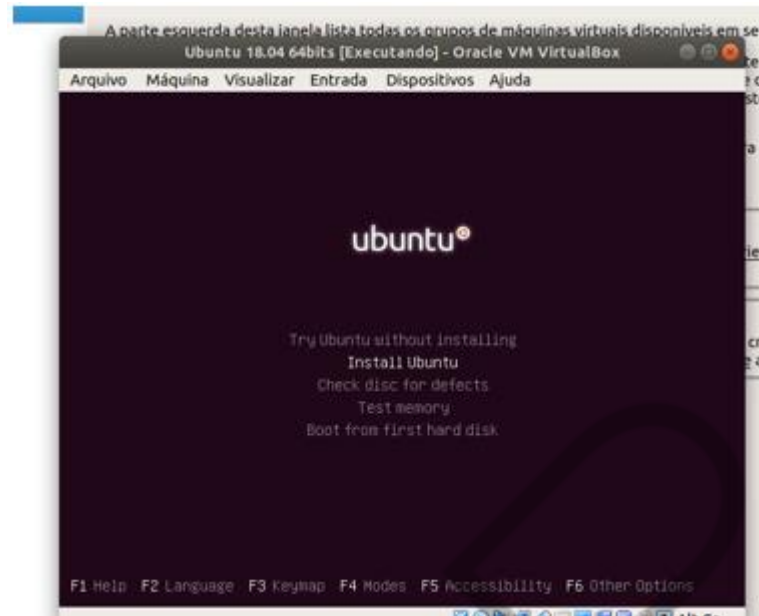


Figura 19 – Configuração da imagem ISO
Fonte: Elaborado pelo autor (2020)

4 SISTEMAS DE NUMERAÇÃO

Um número pode ser representado por um ou mais símbolos, quer seja na história remota quando o homem começou a contar, quer seja atualmente.

Para o processo de criação ou reversão de um software, faz-se necessária a conversão entre o que escrevemos, linguagem humana, geralmente são palavras em inglês (if, for, while), para linguagem de máquina (0110010101), que é justamente o que o processador conseguirá entender e executar. Por exemplo, este documento digital que você está lendo (no computador, celular ou tablet) nada mais é do que números zero e um (01001) organizados em bytes de forma que são processados e apresentados em sua tela.

Então, é de extrema importância que conheçamos os principais sistemas de numeração e conversões entre eles.

4.1 Sistema decimal

No sistema de numeração decimal que usamos hoje, os algarismos são representados pela composição dos dígitos. Como os números são infinitos, os algarismos podem ser compostos por quantos dígitos desejar (MIYASCHITA, 2002).

Atualmente, o sistema de numeração que mais utilizamos é o de base 10 ou sistema decimal, nele podemos representar qualquer número utilizando-se para isso 10 símbolos ou algarismos, a saber:

Símbolos da base decimal: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

É um sistema de numeração posicional, pois o mesmo símbolo/algarismo terá valor diferente no número dependendo de sua posição, como no exemplo a seguir:

$$111 = 100 + 10 + 1$$

Neste caso, o símbolo/algarismo 1 tem seu valor alterado entre 100 (uma centena), 10 (uma dezena) e 1 (uma unidade) diferentemente da numeração romana, na qual utilizamos o símbolo/algarismo “I” para escrever o número “III”, que representa 3 unidades, como podemos observar, cada símbolo tem o mesmo valor independentemente de sua posição.

4.2 Sistema Hexadecimal

Como abordado anteriormente no sistema decimal, este sistema de numeração é também de posição, pois cada símbolo/algarismo tem valores diferentes de acordo com sua posição. Bastante utilizado em matemática e na computação, o hexadecimal, ou apenas “Hexa”, ou também base 16, possui 16 símbolos/algarismos para a representação dos seus números, a saber:

Símbolos da base hexadecimal: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

Como podemos observar, neste sistema os símbolos de “0” – “9” representam os valores 0 (zero unidades) a 9 (9 unidades) e os símbolos “A” – “F” representam de 10 (dez unidades) a 15 (quinze unidades).

Os números em hexa são amplamente utilizados em engenharia reversa e análise da malware, pois fornecem uma representação mais amigável aos valores binários que são extraídos da linguagem de máquina, ou seja, um único byte ou 8 bits (11111111) poderá ser representado com apenas dois dígitos em hexadecimal (FF).

4.3 Sistema Binário

É um sistema numérico posicional de base 2 que consiste em dois diferentes símbolos “0” (zero) e “1” (um), sendo $1 > 0$, para representação de cada numeral. O $1 > 0$ parece óbvio para nós que já conhecemos o símbolo 1 (um) e o símbolo 0 (zero), porém podemos usar qualquer símbolo para representar um número binário ou de base 2, no entanto será necessário inferir qual símbolo será maior e qual será menor para que este sistema de numeração possa valer. Por exemplo, vamos criar um novo sistema de numeração binário posicional no qual teremos dois símbolos  e ∞ sendo $\text{bug} > \infty$.

Agora podemos formar nossos números dentro do sistema de base 2 utilizando-se destes símbolos e convertê-los para decimal.

$$\text{bug} \text{ bug } \infty \infty \infty \infty \text{ bug } \text{ bug } \text{ bug }^2 = 199_{10}$$

Serve de base para Álgebra booleana, permitindo que se façam operações lógicas valendo-se de apenas de 2 dígitos (falso ou verdadeiro) e para eletrônica digital, em que se pode representar o estado de um circuito elétrico ligado ou desligado e quando salvamos nossas fotos, músicas, vídeos e documentos em nossas mídias (Pendrive, HD, SSD, disco etc.) tudo estará armazenado neste sistema de numeração de base 2.

Por ser de fácil implementação em portas lógicas, é amplamente utilizado em dispositivos eletrônicos e baseados em computador.

Entretanto, o sistema de numeração binário tem uma desvantagem em comparação com os demais sistemas numéricos: a enorme quantidade de dígitos que temos que empregar para realizar a notação de números relativamente grandes. Se o sistema de numeração binária fosse a utilizada hoje, o ano de 2002, escrito no sistema decimal, seria denotado por (11111010010)2, por exemplo (MIYASCHITA, 2002).

4.4 Conversão entre bases

Seguindo a tabela “Conversão direta entre as bases apresentadas”, podemos ter uma visão completa e direta de como alternar entre as bases.

HEXADECIMAL	DECIMAL	BINÁRIO
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Tabela 1 – Conversão direta entre as bases apresentadas
Fonte: Elaborado pelo autor (2020)

4.5 Conversão de decimal para binário

A regra de conversão entre decimal e binário é muito simples, devemos representar o número em decimal 25_{10} com uma soma de potência de 2, para isso devemos executar sucessivas divisões por 2 até que o quociente não seja mais possível de ser divisível. O número binário será gerado pelo quociente da última divisão, que representará o bit mais significativo, seguido dos restos das divisões anteriores, como na tabela “Conversão de decimal para binário”.

Ex: $25_{10} = B_2$

$25/2 = 12$, resto **1**

$12/2 = 6$, resto **0**

$$6/2 = 3, \text{ resto } 0$$

$$3/2 = 1, \text{ resto } 1$$

$$25_{10} = 11001_2$$

DIVIDENDO	DIVISOR	QUOCIENTE	RESTO
25	2	12	1
12	2	6	0
6	2	3	0
3	2	1 →	1

Tabela 2 – Conversão de decimal para binário
Fonte: Elaborado pelo autor (2020)

4.6 Conversão de binário para decimal

Para convertermos da base 2 para base decimal, devemos escrever o número binário e multiplicá-los pela base 2 elevado à posição que o bit ocupa, após feita todas as multiplicações devemos somar os resultados e obteremos o número equivalente em decimal, como na figura “Conversão de binário para decimal”.

Para auxiliar na conversão, utilize a “Tabela auxiliar de conversão binário decimal”, nela você deverá preencher a linha que contém a palavra “binário” com o seu número binário até 10 bits; em seguida, faça a multiplicação de cada bit pelo valor logo acima, escrevendo o resultado abaixo, em decimal, e por fim some todos os valores em decimal, o resultado final será o valor em decimal correspondente ao binário.

$$\begin{array}{cccccc}
 1 & 0 & 1 & 0 & 0 & 1 \\
 \times & \times & \times & \times & \times & \times \\
 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 \hline
 32 & + & 0 & + & 8 & + & 0 & + & 0 & + & 1 & = & 41
 \end{array}$$

Figura 20 – Conversão de binário para decimal
Fonte: Elaborado pelo autor

POTÊNCIA	2 ⁹	2 ⁸	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
VALOR	512	256	128	64	32	16	8	4	2	1
BINARIO										
DECIMAL										

Tabela 3 – Tabela auxiliar de conversão binário decimal

Fonte: Elaborado pelo autor (2020)

4.7 Conversão de binário para hexadecimal

A conversão de binário para hexadecimal é bastante simples, devemos lembrar que cada número em hexadecimal equivale a 4 dígitos em binário. Sabendo disso, devemos separar os dígitos binários em grupos de 4 da direita para a esquerda, caso a quantidade de dígitos em binário não seja múltiplo de 4, completaremos com 0 (zeros) a esquerda, como na figura “Conversão de binário para hexadecimal”. Agora, utilize a tabela “Conversão direta entre binário e hexadecimal” para fazer a conversão direta entre cada grupo de binário e seu respectivo hexadecimal.

HEXADECIMAL	BINÁRIO	HEXADECIMAL	BINÁRIO
0	0	8	1000
1	1	9	1001
2	10	A	1010
3	11	B	1011
4	100	C	1100
5	101	D	1101
6	110	E	1110
7	111	F	1111

Tabela 4 – Conversão direta entre binário e hexadecimal

Fonte: Elaborado pelo autor

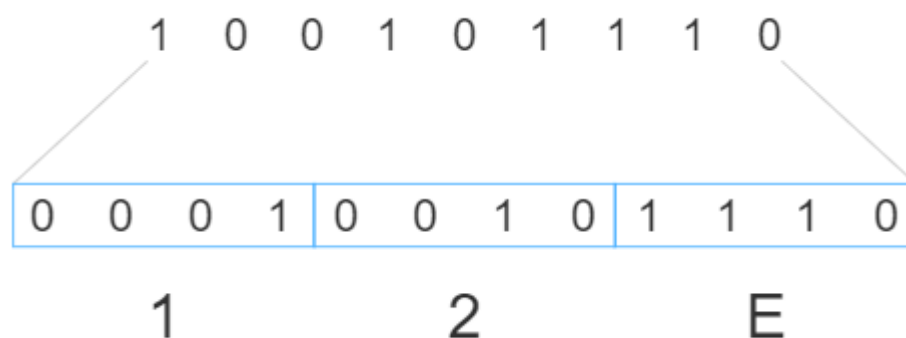


Figura 21 – Conversão de binário para hexadecimal
Fonte: Elaborado pelo autor

Então:

$$100101110_2 = 12E_{16}$$

4.8 Conversão de decimal para hexadecimal

Igualmente simples, como na conversão de decimal para binário, porém, neste caso, a base utilizada será a 16. Para fazermos esta conversão, devemos aplicar sucessivas divisões por 16, como na tabela “Conversão de decimal para hexadecimal”, e se atentar ao fato de que, depois do algarismo 9 em hexa vem a letra A. Devemos observar os valores dos restos e, também, do quociente da última divisão, pois estes serão convertidos diretamente para hexadecimal.

$$61865/16 = 3866, \text{ resto } 9$$

$$3866/16 = 241, \text{ resto } 10$$

$$241/16 = 15, \text{ resto } 1$$

Sabendo-se que 15 e 10 em decimal equivalem às letras F e A em hexadecimal respectivamente, podemos formar o número final em hexa.

$$61865_{10} = F1A9_{16}$$

DIVIDENDO	DIVISOR	QUOCIENTE	RESTO
61865	16	3866	9
3866	16	241	10 (A)
241	16	15 (F) →	1

Tabela 5 – Conversão de decimal para hexadecimal

Fonte: Elaborado pelo autor

5 TABELA ASCII

Computadores só conseguem armazenar informações em formato binário, então na década de 60 criou-se um padrão de códigos que poderia fazer o intercâmbio entre o que nós humanos poderíamos ver e entender e o que o computador, em linguagem de máquina, pudesse armazenar e processar. Um exemplo claro é o seu teclado, nele você pode ver letras (A, a, B, b), números (1337) e outros caracteres (#, \$,), porém estão lá os sinais de controle (\a\b\r\n) também, mesmo que não possamos vê-los.

A este padrão dá-se o nome de ASCII (*American Standard Code for Information Interchange*) ou Código Padrão Americano para Intercâmbio de Informações, em tradução livre para o português. E, nada mais é do que uma tabela que relaciona um número binário de 7 bits 0b0000000 com um caracter imprimível e sinais de controle.

Por exemplo, o caracter “A” é representado em binário por “0b1000001”, em hexadecimal por “0x41” e em decimal por “65” assim como o caracter “a” é representado em binário por “0b1100001”, em hexadecimal por “0x61” e em decimal por “97”. A tabela ASCII é compreendida entre 0b0000000 e 0b1111111 que em decimal vai de 0 a 127.

Uma forma de dado comum é o texto, ou strings de caracteres. Embora os dados textuais sejam mais convenientes para os seres humanos, eles não podem, em forma de caracteres, ser facilmente armazenados ou transmitidos por sistemas de processamento de dados e comunicações. Esses sistemas são projetados para dados binários. Assim, diversos códigos foram elaborados, nos quais os caracteres são representados por uma sequência de bits. Talvez o exemplo comum mais antigo seja o código Morse. Hoje, o código de caracteres mais utilizado é o International Reference Alphabet (IRA), mais conhecido como American Standard Code for Information Interchange (ASCII) (STALLINGS, 2017, p. 358).

Dec	Bin	Hex	Char	Dec	Bin	Hex	Char	Dec	Bin	Hex	Char	Dec	Bin	Hex	Char
0	0000 0000	00	[NUL]	32	0010 0000	20	space	64	0100 0000	40	@	96	0110 0000	60	`
1	0000 0001	01	[SOH]	33	0010 0001	21	!	65	0100 0001	41	A	97	0110 0001	61	a
2	0000 0010	02	[STX]	34	0010 0010	22	"	66	0100 0010	42	B	98	0110 0010	62	b
3	0000 0011	03	[ETX]	35	0010 0011	23	#	67	0100 0011	43	C	99	0110 0011	63	c
4	0000 0100	04	[EOT]	36	0010 0100	24	\$	68	0100 0100	44	D	100	0110 0100	64	d
5	0000 0101	05	[ENQ]	37	0010 0101	25	%	69	0100 0101	45	E	101	0110 0101	65	e
6	0000 0110	06	[ACK]	38	0010 0110	26	&	70	0100 0110	46	F	102	0110 0110	66	f
7	0000 0111	07	[BEL]	39	0010 0111	27	'	71	0100 0111	47	G	103	0110 0111	67	g
8	0000 1000	08	[BS]	40	0010 1000	28	(	72	0100 1000	48	H	104	0110 1000	68	h
9	0000 1001	09	[TAB]	41	0010 1001	29	)	73	0100 1001	49	I	105	0110 1001	69	i
10	0000 1010	0A	[LF]	42	0010 1010	2A	*	74	0100 1010	4A	J	106	0110 1010	6A	j
11	0000 1011	0B	[VT]	43	0010 1011	2B	+	75	0100 1011	4B	K	107	0110 1011	6B	k
12	0000 1100	0C	[FF]	44	0010 1100	2C	,	76	0100 1100	4C	L	108	0110 1100	6C	l
13	0000 1101	0D	[CR]	45	0010 1101	2D	-	77	0100 1101	4D	M	109	0110 1101	6D	m
14	0000 1110	0E	[SO]	46	0010 1110	2E	.	78	0100 1110	4E	N	110	0110 1110	6E	n
15	0000 1111	0F	[SI]	47	0010 1111	2F	/	79	0100 1111	4F	O	111	0110 1111	6F	o
16	0001 0000	10	[DLE]	48	0011 0000	30	0	80	0101 0000	50	P	112	0111 0000	70	p
17	0001 0001	11	[DC1]	49	0011 0001	31	1	81	0101 0001	51	Q	113	0111 0001	71	q
18	0001 0010	12	[DC2]	50	0011 0010	32	2	82	0101 0010	52	R	114	0111 0010	72	r
19	0001 0011	13	[DC3]	51	0011 0011	33	3	83	0101 0011	53	S	115	0111 0011	73	s
20	0001 0100	14	[DC4]	52	0011 0100	34	4	84	0101 0100	54	T	116	0111 0100	74	t
21	0001 0101	15	[NAK]	53	0011 0101	35	5	85	0101 0101	55	U	117	0111 0101	75	u
22	0001 0110	16	[SYN]	54	0011 0110	36	6	86	0101 0110	56	V	118	0111 0110	76	v
23	0001 0111	17	[ETB]	55	0011 0111	37	7	87	0101 0111	57	W	119	0111 0111	77	w
24	0001 1000	18	[CAN]	56	0011 1000	38	8	88	0101 1000	58	X	120	0111 1000	78	x
25	0001 1001	19	[EM]	57	0011 1001	39	9	89	0101 1001	59	Y	121	0111 1001	79	y
26	0001 1010	1A	[SUB]	58	0011 1010	3A	:	90	0101 1010	5A	Z	122	0111 1010	7A	z
27	0001 1011	1B	[ESC]	59	0011 1011	3B	;	91	0101 1011	5B	[	123	0111 1011	7B	{
28	0001 1100	1C	[FS]	60	0011 1100	3C	<	92	0101 1100	5C	\	124	0111 1100	7C	
29	0001 1101	1D	[GS]	61	0011 1101	3D	=	93	0101 1101	5D	]	125	0111 1101	7D	}
30	0001 1110	1E	[RS]	62	0011 1110	3E	>	94	0101 1110	5E	^	126	0111 1110	7E	~
31	0001 1111	1F	[US]	63	0011 1111	3F	?	95	0101 1111	5F	_	127	0111 1111	7F	[DEL]

Figura 22 – Tabela ASCII convertida em decimal, binário e hexa
 Fonte: Google Imagens (2020)

Vamos ver alguns exemplos de conversões desta tabela ASCII no Shell do Linux no Comando de prompt “Utilizando o comando echo e o comando de hd do Linux” para verificar a conversão entre os caracteres da tabela ASCII e o hexadecimal e decimal equivalente.

COMANDOS

```
echo "MALWARE ANALYSIS AND THREAT PROTECTION" | hd
```

Comando de prompt 4 – Utilizando o comando echo e o comando de hd do Linux

Fonte: Elaborado pelo autor (2020)

Como podemos observar, toda nossa string foi convertida em hexadecimal, letra a letra - ao lado esquerdo temos os códigos em hexadecimal e à direita temos os caracteres imprimíveis.

IMPORTANTE: No final da penúltima linha, |ECTION.| existe um “.”, verifique na tabela ASCII qual é o código relacionado a este caractere e tente entender o porquê de o código “0a” estar relacionado a ele.

5.1 ASCII estendida

Originalmente, o ASCII foi projetado com um código de 7 bits (de 0 a 127), que continha todos os caracteres de texto do idioma americano, local de sua criação. Para dar suporte a idiomas estrangeiros, foi adicionado mais 1 bit aos códigos da tabela ASCII, que forneceu mais 128 adicionais e ficou conhecido como *extended* ASCII. Os códigos do *extended* ASCII utilizam 1 byte (8 bits) que vai de 0 a 255.

0 a 127: ASCII.

128 a 255: *Extended* ASCII.

6 ARQUIVOS

Um arquivo é um recurso de computador utilizado para armazenar dados e sequência de bytes, diretamente em dispositivos de armazenamento e são organizados em um sistema de arquivos para que possam ser localizados, abertos, lidos, alterados, salvos e fechados infinitas vezes.

Em sua forma mais simples, um arquivo consiste em uma sequência de bytes escrita em um dispositivo de E/S. Se este for um dispositivo de armazenamento, como um disco, o arquivo pode ser lido de volta mais tarde; se o dispositivo não for de armazenamento (por exemplo, uma impressora), não pode ser lido de volta, é claro (TANENBAUM, 2013, p. 367)

Existem diversos tipos de arquivos, porém nem sempre conseguimos identificá-los apenas por suas extensões (.txt, .pdf, .xls etc.) como vemos no comando de prompt “Levantando informações sobre um arquivo”, onde:

COMANDOS

```
touch mydocs.pdf
```

```
echo -n "FIAP ON" > mydocs.pdf
```

```
file mydocs.pdf
```

```
hd mydocs.pdf
```

```
stat mydocs.pdf
```

Comando de prompt 5 – Levantando informações sobre um arquivo
Fonte: Elaborado pelo autor (2020)

- Criamos um arquivo em branco utilizando o comando touch do Linux e demos o nome de mydocs.pdf.
- Adicionamos um conteúdo ao arquivo criado com o comando echo.
- Podemos observar algumas características e informações interessantes sobre o arquivo com os comandos file, hd e stat.
- Mesmo que o arquivo tenha a extensão .pdf, o comando file nos indica que é um arquivo de texto ASCII e não tem o delimitador de linha (espero que tenha descoberto o que é o 0a).
- O comando hd nos mostra o conteúdo do arquivo em hexadecimal, além disso, temos outra informação importante: Observem a última linha "00000007", ela nos indica o tamanho do arquivo, 7 bytes, basta contar os bytes da primeira linha.
- O comando stat, além de confirmar o tamanho do arquivo "Size: 7", nos mostra informações de último acesso, data de criação e dono e grupo a que o arquivo pertence, permissões do arquivo e Inode.

Agora vamos criar um arquivo no LibreOffice com o mesmo conteúdo do arquivo criado anteriormente, "FIAP ON"; depois, exportaremos para mydocs2.pdf e no comando de prompt "Levantando Informações do arquivo mydocs2.pdf" executaremos os mesmos comandos para comparar as saídas.

COMANDOS

```
file mydocs2.pdf
```

```
hd mydocs2.pdf
```

```
stat mydocs2.pdf
```

Comando de prompt 6 – Levantando Informações do arquivo mydocs2.pdf

Fonte: Elaborado pelo autor (2020)

Como podemos ver, o arquivo mydocs2.pdf gerado pelo LibreOffice é consideravelmente maior que o arquivo gerado anteriormente, convertendo a última linha da saída do comando hd (00001bcb) para decimal, veremos que corresponde ao mesmo valor do parâmetro Size: 7115 do comando stat e que é 1000 vezes maior que o arquivo anterior, mesmo os dois arquivos contendo a mesma frase. Isso

acontece porque o arquivo mydocs2.pdf, além de conter o texto, pode conter imagens, pode ser protegido e até executar Java Script, então é criada toda uma estrutura dentro do arquivo para isso.

Contudo, podemos enganar ou sermos enganados por arquivos, mesmo que o comando file nos mostre que o tipo de arquivo não seja exatamente o que estamos vendo.

Então, como o comando file funciona?

Para que o comando file funcione, ele precisa identificar a assinatura de cada arquivo, conhecida também como Magic Number ou Magic Bytes, essas assinaturas são dados que identificam cada arquivo como visto na primeira linha do comando `hd mydocs2.pdf` no comando de prompt “Levantando Informações do arquivo mydocs2.pdf”.

O comando file usa um arquivo chamado magic.mgc, que contém todas os Magic Numbers para todo tipo de arquivo conhecido e está localizado em `/usr/lib/file/` como no comando de prompt “Localização do arquivo magic.mgc”.

COMANDOS

```
ls -lh
```

Comando de prompt 7 – Localização do arquivo magic.mgc
Fonte: Elaborado pelo autor (2020)

Sabendo como o file funciona, e como ele verifica os arquivos, vamos transformar o arquivo mydocs.pdf, que é originalmente um arquivo de texto, em um arquivo .pdf, igualmente ao arquivo mydocs2.pdf gerado pelo LibreOffice; depois, repetiremos o processo e o transformaremos em um arquivo executável do Windows.

Para fazer este procedimento, faz-se necessário um editor de hexadecimal. Em nosso caso, utilizaremos o hexex, uma ótima opção para quem estiver familiarizado com o editor de texto VI. Para instalá-lo, utilize o comando de prompt “Instalação do hexer”, editor de arquivo hexadecimal.

COMANDOS

```
apt install hexer
```

Comando de prompt 8 – Instalação do hexer
Fonte: Elaborado pelo autor (2020)

- Para descobrir o Magic Number de um arquivo .pdf, no nosso caso, utilizaremos o mydocs2.pdf, e copiaremos os primeiros 8 bytes da linha marcada no comando de prompt “Magic number de um arquivo .pdf”.

COMANDOS

```
hd mydocs2.pdf | head -n 5
```

Comando de prompt 9 – Magic number de um arquivo .pdf
Fonte: Elaborado pelo autor (2020)

- Abrir o arquivo mydocs.pdf: É um arquivo de texto puro, com o editor hexer adicionaremos os bytes copiados dentro do arquivo.
- Conferindo o tipo de arquivo e o seu conteúdo.

COMANDOS

```
file mydocs.pdf  
cat mydocs.pdf
```

Comando de prompt 10 – Conferindo o arquivo antes da edição
Fonte: Elaborado pelo autor (2020)

- Abrindo o arquivo com o hexer para manipulação dos hexadecimais.

COMANDOS

```
hexer mydocs.pdf
```

Comando de prompt 11 – Abrindo o arquivo mydocs.txt editor de hexadecimal
Fonte: Elaborado pelo autor (2020)

- Arquivo original mydocs.pdf (lembrem-se de que é um arquivo .txt e apenas com a extensão trocada).

```
00000000: 46 49 41 50 20 4f 4e -- -- -- -- -- -- -- -- FIAP ON-----
```

Comando de prompt 12 – Edição do arquivo mydocs.pdf com o hexer
Fonte: Elaborado pelo autor (2020)

- Arquivo após alteração.

```
00000000: 25 50 44 46 2d 31 2e 34 46 49 41 50 20 4f 4e -- %PDF-1.4FIAP ON-
```

Comando de prompt 13 – Adição do Magic number ao arquivo mydocs.pdf
Fonte: Elaborado pelo autor (2020)

- Verificando com o comando file, o tipo de arquivo que acabamos de editar, como podemos observar, para o comando file, é um arquivo pdf.

COMANDOS

```
file mydocs.pdf
```

Comando de prompt 14 – Arquivo de texto mascarado em um pdf
Fonte: Elaborado pelo autor (2020)

Agora vamos transformar este arquivo, mydocs.pdf, em um inofensivo arquivo executável do MS-DOS, mas, com algumas técnicas, podemos transformá-lo também em um arquivo malicioso.

- Primeiro baixe um arquivo .exe de sua preferência, no meu caso baixei o vulnserver.exe no github (este arquivo é utilizado para desenvolvimento de exploits e é totalmente vulnerável a ataques remotos. Eu o desaconselho executá-lo em seu sistema operacional).
- Verificando o arquivo baixado, percebe-se que o arquivo tem o Magic Number MZ, porém a assinatura completa é terminada em PE, para indicar um executável do MS-DOS, como no comando de prompt “Assinatura de um arquivo executável”. "MZ" são as iniciais de Mark Zbikowski, um dos principais desenvolvedores do MS-DOS.

COMANDOS

```
hd vulnserver.exe | head
```

Comando de prompt 15 – Assinatura de um arquivo executável
Fonte: Elaborado pelo autor (2020)

- Copiar toda assinatura do arquivo executável para inserir dentro do arquivo mydocs.pdf. Com o comando abaixo, podemos filtrar a saída que será copiada e colada dentro do nosso arquivo mydocs.pdf.

COMANDOS

```
hd vulnserver.exe | head -9 | cut -d " " -f 2-19
```

Comando de prompt 16 – Assinatura filtrada do arquivo executável
Fonte: Elaborado pelo autor (2020)

- Após copiada a saída, que encontra-se no comando acima, vamos editar o arquivo mydocs.pdf e adicionar este conteúdo dentro do arquivo, com o editor hexer, como no comando de prompt abaixo.

```

00000000: 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00 MZ.....
00000010: b3 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 .....@.....
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000030: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000040: 0e 1f ba 0e 00 b4 09 cd 21 b3 01 4c cd 21 54 63 .....!.L!Th
00000050: 69 73 20 70 72 6f 67 72 61 6d 20 63 61 6e 6e 6f is program canno
00000060: 74 20 62 65 20 72 75 6e 20 69 6e 20 44 4f 53 20 t be run in DOS
00000070: 6d 6f 64 65 2e 0d 0d 0a 24 00 00 00 00 00 00 00 mode...$.
00000080: 50 45 46 49 41 50 20 4f 4e -- -- -- -- -- -- -- PEFIAP ON-----

```

Comando de prompt 17 – Inserção do magic number executável
 Fonte: Elaborado pelo autor (2020)

Concluída a edição do arquivo, utilizaremos o comando file e o site <www.virustotal.com> para identificar o arquivo que acabamos de modificar, como mostrado nos comandos abaixo.

COMANDOS

```
sha256sum mydocs.pdf
```

```
file mydocs.pdf
```

Comando de prompt 18 – Verificação do arquivo mydocs.pdf no linux
 Fonte: Elaborado pelo autor (2020)

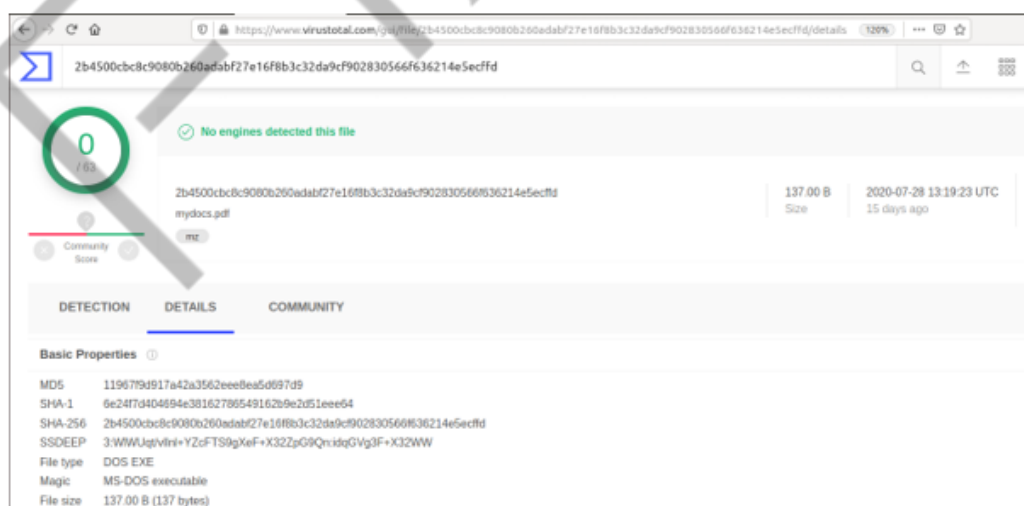


Figura 23 – Verificação do arquivo mydocs.pdf no vírus total
 Fonte: Elaborado pelo autor (2020)

Como podemos observar, temos agora um arquivo com extensão .pdf, e seu conteúdo é um simples e inofensivo texto da FIAP ON, reconhecido como um executável do MS-DOS, tanto pelo comando file como no <www.virustotal.com>.

CONCLUSÃO

Neste capítulo introdutório, iniciamos nossa preparação de ambiente com a instalação e configuração do Virtualbox, preparando-o para a instalação das VMs Ubuntu e Windows. Esta preparação inicial e instalação da VM Ubuntu foi necessária, pois utilizamos algumas ferramentas do Linux para dar continuidade às tarefas dos capítulos bem como executar todo e qualquer exercício diretamente na VM a partir deste capítulo.

É de extrema importância que o aluno conheça bem os tipos de sistemas de numeração e suas conversões, pois é comum aparecer determinadas strings em malwares que, só em vê-las, já saberemos em que base se encontra; e em instruções do processador (opcodes), que usaremos em análise estática avançada, encontraremos o sistema de numeração hexadecimal.

A análise inicial de qualquer malware será sempre uma análise estática, pois a partir deste ponto, poderemos saber por onde devemos caminhar. Não adianta você receber um malware que foi coletado dentro de uma máquina Linux, preparar todo um ambiente Linux, e o malware ter sido desenvolvido para Windows. Com isto, neste capítulo, aprendemos como analisar, mesmo que de forma básica, um arquivo por meio de ferramentas no Linux e na internet.

REFERÊNCIAS

MIYASCHITA, W. Y. **Sistemas de Numeração** – Como funcionam e como são estruturados os números. 2002. Disponível em: <<http://www.fc.unesp.br/~mauri/TN/SistNum.pdf>>. Acesso em: 06 jul. 2020.

STALLINGS, W. **Arquitetura e organização de computadores**. São Paulo: Pearson, 2017.

TANENBAUM, A. S. **Organização estruturada de computadores**. São Paulo: Person, 2013.

TANENBAUM, A. S.; BOS, H. **Sistemas Operacionais Modernos**. São Paulo: Pearson, 2016.